

Interfaces graphiques



Dr Khadim DRAME*
kdrame@univ-zig.sn

16 janvier 2024

Table des matières

1	Introduction	2
2	Fenêtres	3
2.1	JFrame	3
2.2	JDialog	4
2.3	JOptionPane	4
2.4	JFileChooser	5
3	Conteneurs	5
3.1	JPanel	6
3.2	JScrollPane	6
3.3	JTabbedPane	6
3.4	JSplitPane	7
4	Composants atomiques	7
4.1	Boutons	7
4.2	Composants de saisie de textes	8
4.3	Liste de choix	8
4.4	Tableaux	9
4.5	Menus	9
5	Gestionnaire de dispositions	10
5.1	BorderLayout	11
5.2	FlowLayout	11
5.3	GridLayout	11
5.4	CardLayout	11

*UFR Sciences et Technologies, Université Assane Seck de Ziguinchor

1 Introduction

Une interface graphique (Graphical User Interface ou GUI) est une façade du programme qui le lie à l'extérieur. La plupart des logiciels disposent d'une interface graphique. Java fournit des composants pour réaliser des interfaces graphiques. Deux packages sont principalement utilisés : **AWT** (Abstract Window Toolkit) et **Swing** (basé sur awt). Lors des premières versions du langage Java, seul le package `java.awt` était fourni pour créer des interfaces graphiques. Aujourd'hui, il est fortement recommandé d'utiliser le package `javax.swing` qui a été intégré au JDK depuis sa version 1.2. Swing offre plusieurs avantages :

- il est simple et multi-plateforme ;
- il a de bonnes performances ;
- il dispose d'un ensemble riche de composants graphiques légers ;
- il permet de réaliser des interfaces graphiques complexes ;
- il utilise l'architecture MVC (Modèle-Vue-Contrôleur).

La plupart des classes de composants du package `javax.swing` héritent, directement ou indirectement, de classes du package `java.awt` et redéfinissent certaines méthodes. Cependant, le package `java.awt` n'est pas entièrement obsolète, il est encore utilisé par exemples pour la gestion de la disposition des composants graphiques dans un conteneur (classes implémentant l'interface `LayoutManager`) et des des événements.

Dans une GUI, les objets sont généralement imbriqués en trois niveaux

- le **conteneur de niveau supérieur** qui permet d'encapsuler les entités des 2 autres niveaux (par exemple `JFrame`) ;
- le **conteneur intermédiaire** qui permet de regrouper en son sein des composants atomiques (par exemple `JPanel`) ;
- le **composant atomique** qui constitue l'élément de base (boutons, zones de saisie, liste déroulante, etc.).

Les composants Swing héritent de la classe `JComponent` (voir Figure 1), à l'exception des conteneurs de niveau supérieur. La classe `JComponent` hérite elle même de la classe `Container` qui hérite de `Component`.

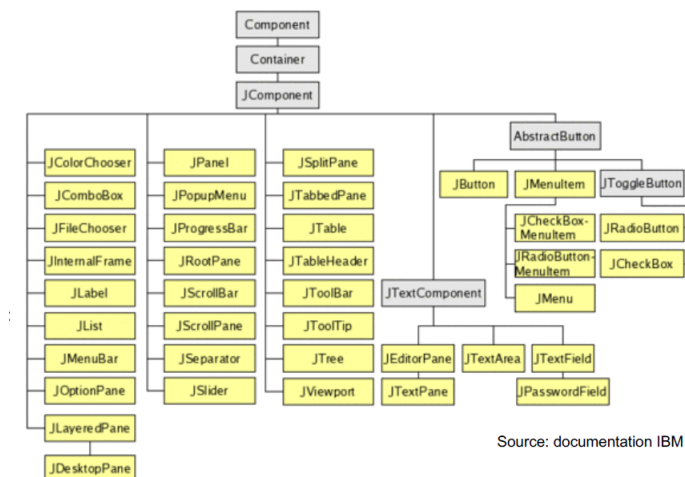


FIGURE 1 – Composants de Swing

2 Fenêtres

Swing fournit plusieurs classes permettant de construire des fenêtres dont les principales sont : `JFrame`, `JDialog`, `JOptionPane`, `JFileChooser`, `JColorChooser`.

2.1 JFrame

La classe `JFrame` permet de créer une fenêtre principale possédant une barre de titre et une bordure. Elle définit de nombreuses méthodes qui peuvent être utilisées pour personnaliser une fenêtre. La fenêtre créée peut être agrandie, diminuée ou dimensionnée.

La classe `JFrame` dispose de plusieurs constructeurs pour créer une fenêtre.

```
1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 public class TestJFrame{
4     public static void main (String args[]){
5         JFrame fenetre = new JFrame() ;
6         fenetre.setSize(300, 200) ;//
7         fenetre.setTitle("Ma première fenêtre") ;// donner un titre à la fenêtre
8         fenetre.setVisible(true) ;
9         // rendre la fenêtre visible. Par défaut, elle n'est pas visible
10        fenetre.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
11        // fermer la fenêtre quand on clique sur la croix rouge
12    }
13 }
```

Listing 1 – Création de fenêtre avec `JFrame`

Pour créer une fenêtre, on peut aussi étendre la classe `JFrame`.

```
1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 public class MaFenetre extends JFrame{
4     public MaFenetre (){
5         setTitle("Ma première fenêtre") ;
6         setSize(300, 200) ;
7     }
8     public static void main(String[] args) {
9         JFrame fenetre = new MaFenetre() ;
10        // créer une fenêtre avec la classe MaFenetre
11        fenetre.setVisible(true) ; // rendre visible la fenêtre
12    }
13 }
```

Listing 2 – Création de fenêtre avec une classe étendant `JFrame`

Méthodes communément utilisées par les `JFrame` :

- `add(Component)` : ajoute un composant dans le `JFrame` ;
- `setLocation(int x, int y)` : change la position de la fenêtre à l'écran. Le coin supérieur gauche de l'écran a pour position (0,0) ;
- `setSize(int longueur, int largeur)` : fixe la taille de la fenêtre (en pixels) ;
- `setBounds(int x, int y, int longueur, int largeur)` : positionne la fenêtre sur l'écran et fixe sa taille ;
- `setTitle(String)` : spécifie le titre de la fenêtre ;
- `setResizable(boolean)` : la valeur **false** permet de désactiver le redimensionnement de la taille de la fenêtre ;
- `setLayout(LayoutManager)` : gère la disposition des composants du `JFrame` ;

- `setVisible(boolean)` : la valeur **true** permet de rendre la fenêtre visible. Par défaut, la fenêtre n'est pas visible ;
- `setDefaultCloseOperation(int)` : définit ce qui se produira lorsque l'utilisateur clique sur la "croix rouge".

La méthode `setDefaultCloseOperation(int)` peut prendre en argument :

- `JFrame.EXIT_ON_CLOSE` : on quitte l'application et libère la mémoire ;
- `JFrame.DISPOSE_ON_CLOSE` : on ferme la fenêtre mais elle continue de fonctionner ;
- `JFrame.HIDE_ON_CLOSE` : la fenêtre est simplement masquée sans être fermée ;
- `JFrame.DO_NOTHING_ON_CLOSE` : rien n'est fait lorsque l'utilisateur clique sur la "croix rouge".

2.2 JDialog

La classe `JDialog` permet de créer une fenêtre secondaire dépendante d'une fenêtre principale. Cette fenêtre apparaît généralement au dessus d'une autre fenêtre. Elle est souvent temporaire et **modale**, c'est à dire il n'est pas possible d'interagir avec la fenêtre mère tant que la boîte de dialogue est ouverte.

2.3 JOptionPane

La classe `JOptionPane` fournit des boîtes de dialogue standards. Elle sert à afficher des informations ou recueillir des informations de l'utilisateur. On distingue trois types de boîtes de dialogue :

- une boîte de dialogue pour afficher un message d'information :
`JOptionPane.showMessageDialog()` ;
- une boîte de dialogue de confirmation (avec Yes, No et Cancel) :
`JOptionPane.showConfirmDialog()` ;
- une boîte de dialogue pour saisir de données :
`JOptionPane.showInputDialog()` (avec OK et Cancel).

```

1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 public class MessageDialog{
4     public static void main (String args[]){
5         JOptionPane.showMessageDialog(null, "Bienvenue au cours de Java 2");
6         JOptionPane.showMessageDialog(null, "Mise à jour avec succès !", "Attention
7     }, JOptionPane.WARNING_MESSAGE);
8 }

```

Listing 3 – Création de boîtes de dialogue d'information avec `JOptionPane`

```

1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 public class ConfirmInputDialog{
4     public static void main (String args[]){
5         int n = JOptionPane.showConfirmDialog(null, "Voulez vous vraiment supprimer
6         ce membre?", "Suppression", JOptionPane.YES_NO_OPTION);
7         String name = JOptionPane.showInputDialog(null, "Donner votre nom ", "
        Saisie du nom ", JOptionPane.OK_CANCEL_OPTION);
8     }

```

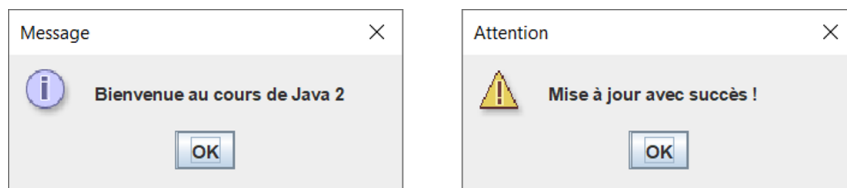


FIGURE 2 – Exemples de messages d’information

8 }

Listing 4 – Création de boîtes de dialogue de confirmation et de saisie avec JOptionPane

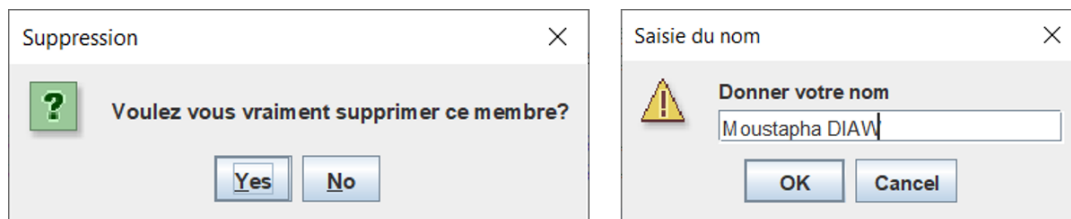


FIGURE 3 – Exemples de boîtes de confirmation et de saisie

2.4 JFileChooser

La classe `JFileChooser` permet de sélectionner un fichier en parcourant l’arborescence du système de fichiers.

```

1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 import java.io.File;
4 public class TestJFileChooser {
5     public static void main(String[] args) {
6         JFrame fen = new JFrame();
7         fen.setSize(300, 200);
8         JFileChooser fc = new JFileChooser();
9         int val = fc.showOpenDialog(fen);
10        if (val == JFileChooser.APPROVE_OPTION){
11            File fichier = fc.getSelectedFile();
12            System.out.println(fichier.getName());
13            System.out.println(fichier.getAbsolutePath());
14        }
15    }
16 }
```

Listing 5 – Exemple d’utilisation de `JFileChooser`

3 Conteneurs

Un **conteneur** est un composant qui contient et gère d’autres composants graphiques. Il sert à placer des composants dans une fenêtre. Il est fondamental dans la

conception d'interface graphique. La classe `JFrame` dispose d'une méthode `getContentPane()` renvoyant un conteneur. Swing fournit plusieurs classes définissant des conteneurs : `JPanel`, `JScrollPane`, `JTabbedPane`, `JSplitPane`.

Méthodes couramment utilisées par les conteneurs :

- `add(Component)` pour ajouter un composant dans le conteneur ;
- `setSize(int longueur, int largeur)` pour fixe la taille du conteneur ;
- `setBounds(int x, int y, int longueur, int largeur)` pour positionner le conteneur et fixer sa taille ;
- `setLocation(int x, int y)` pour changer la position du conteneur à l'écran ;
- `setLayout(LayoutManager)` pour gérer la disposition des composants du conteneur ;

Les conteneurs peuvent être emboîtés, particulièrement les `JPanel`.

3.1 JPanel

`JPanel` permet de contenir d'autres composants. Il permet aussi d'afficher ou de dessiner des images. La classe `JPanel` sert à organiser les composants en spécifiant leur disposition.

```
1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 public class TestJPanel{
4     public static void main (String args[]){
5         JFrame fen = new JFrame() ;
6         fen.setTitle ("Ma fenêtre avec JPanel") ;
7         JPanel p = new JPanel();
8         p.add(new JButton("Bouton 1"));
9         p.add(new JButton("Bouton 2"));
10        p.add(new JButton("Bouton 3"));
11        p.setBounds(40,50,200,150);
12        p.setBackground(Color.lightGray);
13        fen.add(p);
14        fen.setSize(350,300);
15        fen.setLayout(null);
16        fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
17        fen.setVisible(true);
18    }
19 }
20 }
```

Listing 6 – Exemple de conteneur `JPanel`

3.2 JScrollPane

La classe `JScrollPane` permet d'avoir des barres de défilement lorsque le composant qu'il contient est trop grand par rapport à la taille qui lui est allouée.

3.3 JTabbedPane

La classe `JTabbedPane` permet d'avoir plusieurs panneaux sur la même surface en utilisant le système des onglets.

```
1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 public class TestJTabbedPane{
4     public static void main (String args[]){
```

```

5      JFrame fen = new JFrame() ;
6      fen.setSize (300, 200) ;
7      fen.setTitle ("Ma fenêtre avec onglets" ) ;
8      fen.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
9      fen.setVisible(true);
10     JTabbedPane tab = new JTabbedPane();
11     tab.addTab("Onglet 1", new JLabel("Panneau 1"));
12     tab.addTab("Onglet 2", new JLabel("Panneau 2"));
13     tab.addTab("Onglet 3", new JLabel("Panneau 3"));
14     fen.setContentPane(tab);
15 }
16 }

```

Listing 7 – Exemple d'utilisation de JScrollPane

3.4 JSplitPane

La classe `JSplitPane` permet de diviser un panneau en deux composants. Les deux composants peuvent être alignés de gauche à droite ou de haut à bas.

4 Composants atomiques

Les composants atomiques héritent de la classe `JComponent`. Ils sont constitués de :

- Boutons : `JButton`, `JRadioButton`, `JCheckBox`
- Textes : `JLabel`, `TextField`, `PasswordField`
- Listes de choix : `JList`, `ComboBox`
- ...

4.1 Boutons

- `JButton` est utilisée pour créer un bouton dans une interface. La classe `JButton` hérite de la classe `AbstractButton`.
- `JCheckBox` représente une case à cocher. Une case à cocher est indépendante d'une autre.
- `JRadioButton` permet de créer un bouton radio. Le bouton radio est utilisé pour sélectionner une option parmi plusieurs.



FIGURE 4 – Types de boutons

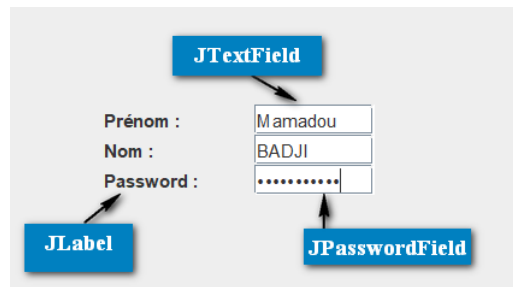


FIGURE 5 – Etiquettes et zones de saisie

4.2 Composants de saisie de textes

- JTextField représente une zone de saisie de textes sur une seule ligne.
- JTextArea représente une zone de saisie de textes sur plusieurs lignes.
- JPasswordField permet la saisie de mots de passe.
- JLabel permet d'afficher un texte ou une image. Il peut contenir plusieurs lignes.

```

1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 public class FenetreAvecComposants{
4     public static void main (String args[]){
5         JFrame frame = new JFrame("Interface de connexion");
6         JLabel label1 = new JLabel("Nom utilisateur ");
7         label1.setBounds(20, 20, 100, 30);
8         JLabel label2 = new JLabel("Mot de passe ");
9         label2.setBounds(20, 70, 100, 30);
10        JTextField tf = new JTextField();
11        tf.setBounds(120, 20, 160, 30);
12        JButton btn = new JButton("Se connecter");
13        btn.setBounds(120, 130, 110, 30);
14        JPasswordField password = new JPasswordField();
15        password.setBounds(120, 70, 160, 30);
16        frame.add(label1); frame.add(label2);
17        frame.add(tf); frame.add(password);
18        frame.add(btn);
19        frame.setSize(400,220);
20        frame.setLayout(null);
21        frame.setVisible(true);
22    }
23 }

```

Listing 8 – Exemple de fenêtre avec des composants atomiques

4.3 Liste de choix

JList propose une liste d'éléments à choisir, rangés en colonne. On peut sélectionner un ou plusieurs éléments. Un JList est souvent contenu dans un JScrollPane.

```

1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 public class ListeElements {
4     public static void main(String[] args) {
5         JFrame fen = new JFrame("Fenêtre avec liste de choix");
6         String langages[] = {"C", "Java", "Lisp", "Python", "Pascal", "Caml"};
7         JList<String> list = new JList<String>(langages);
8         list.setBounds(50,10,75,120);
9         fen.add(list);
10        fen.setSize(400,300);

```



```

11     fen.setLayout(null);
12     fen.setVisible(true);
13 }
14 }

```

Listing 9 – Exemple de liste de choix avec JList

JComboBox (liste déroulante) propose un menu sous forme d'une liste permettant à l'utilisateur de sélectionner un choix.

```

1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 public class ListeDeroulante {
4     public static void main(String[] args) {
5         JFrame fen = new JFrame("Fenêtre avec liste déroulante");
6         String pays[] = {"Sénégal", "Gambie", "Guinée", "Mali", "Cote d'Ivoire", "Nigé-
7             ria"};
8         JComboBox<String> cb = new JComboBox<String>(pays);
9         cb.setBounds(50,10,140,20);
10        fen.add(cb);
11        fen.setLayout(null);
12        fen.setSize(300,200);
13        fen.setVisible(true);
14    }
15 }

```

Listing 10 – Exemple de liste déroulante avec JComboBox

4.4 Tableaux

La classe JTable est utilisée pour afficher/modifier des données sous forme d'un tableau à deux dimensions.

```

1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 public class Tableau {
4     public static void main(String[] args) {
5         JFrame fen = new JFrame("Tableau de personnes");
6         String data[][] = {{ "101", "Fatou", "Diop"}, {"102", "Lamine", "Bâ"},
7             {"103", "Modou", "Faty"}, {"104", "Ndéye Fatou", "Sall"} };
8         String column[] = {"Numéro", "Prénom", "Nom"};
9         JTable table = new JTable(data, column);
10        table.setBounds(30,40,200,300);
11        JScrollPane sp = new JScrollPane(table);
12        fen.add(sp);
13        fen.setSize(400,200);
14        fen.setVisible(true);
15    }
16 }

```

Listing 11 – Exemple d'utilisation de JTable

4.5 Menus

- La classe JMenuBar est utilisée pour créer une barre de menus dans une fenêtre. Cette barre peut contenir plusieurs menus.
- La classe JMenu permet de créer un menu déroulant (JMenuBar).
- La classe JMenuItem permet de définir les éléments d'un menu.

```

1 package sn.uasz.m1.gui;
2 import javax.swing.* ;
3 public class Menu {
4     public static void main(String[] args) {

```

```

5 JFrame fen = new JFrame();
6 JMenuBar menubar = new JMenuBar();
7 JMenuItem nouveau = new JMenuItem("Nouveau");
8 JMenuItem ouvrir = new JMenuItem("Ouvrir");
9 JMenuItem enregistrer = new JMenuItem("Enregistrer");
10 JMenuItem renommer = new JMenuItem("Renommer");
11 JMenuItem cut = new JMenuItem("Couper");
12 JMenuItem copy = new JMenuItem("Copier");
13 JMenuItem paste = new JMenuItem("Coller");
14 JMenuItem selectAll = new JMenuItem("Sélectionner tout");
15 JMenu file = new JMenu("Fichier");
16 JMenu edit = new JMenu("Edition");
17 JMenu help = new JMenu("Aide");
18 file.add(nouveau); file.add(ouvrir); file.add(enregistrer); file.add(renommer);
19 edit.add(cut); edit.add(copy); edit.add(paste); edit.add(selectAll);
20 menubar.add(file); menubar.add(edit); menubar.add(help);
21 JTextArea text = new JTextArea();
22 text.setBounds(5,5,360,200);
23 fen.add(menubar); fen.add(text);
24 fen.setJMenuBar(menubar);
25 fen.setLayout(null);
26 fen.setSize(400,200);
27 fen.setVisible(true);
28 }
29 }

```

Listing 12 – Exemple d'utilisation d'un menu

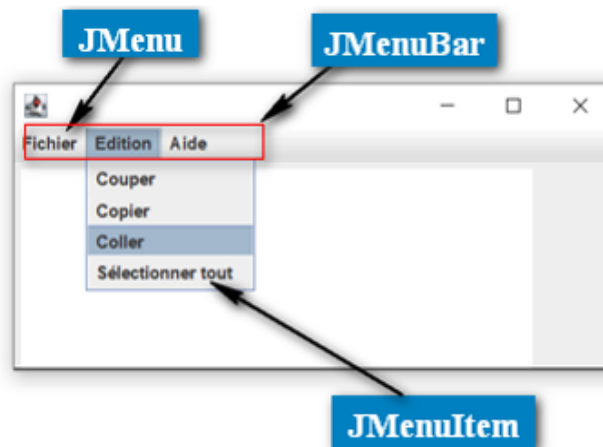


FIGURE 6 – Menu et ses composants

5 Gestionnaire de dispositions

Un gestionnaire de dispositions (appelé **layout manager**) assure la disposition spatiale des composants d'un conteneur qu'il calcule automatiquement. Chaque conteneur est associé à un layout manager qui peut être changé par la méthode `setLayout()`.

L'utilisation d'un layout manager offre plusieurs avantages : pas de calculs compliqués des positions des composants, configuration facile, adaptabilité des composants.

On distingue plusieurs types de gestionnaires : `BorderLayout`, `FlowLayout`, `GridLayout`, `CardLayout`, `GridBagLayout`, `BoxLayout`, `SpringLayout`, etc. Ces classes implémentent l'interface `LayoutManager` du package `java.awt`.

5.1 BorderLayout

`BorderLayout` permet d'arranger les composants du conteneur dans cinq zones : North (Nord), South (Sud), East (Est), West (Ouest) et Center (Centre). C'est la disposition utilisée par défaut dans les fenêtres (`JFrame`, `JDialog`).



FIGURE 7 – Exemple avec BorderLayout

5.2 FlowLayout

`FlowLayout` permet de ranger les composants ligne par ligne, les uns après les autres. C'est la disposition utilisée par défaut dans les panneaux (`JPanel`).



FIGURE 8 – Exemple avec FlowLayout

5.3 GridLayout

`GridLayout` permet de ranger les composants sur une grille rectangulaire. Le conteneur est divisé en cellules de même taille.

5.4 CardLayout

`CardLayout` gère les composants en sorte qu'un seul composant soit visible à la fois. Chaque composant est considéré comme une carte. Il permet de limiter

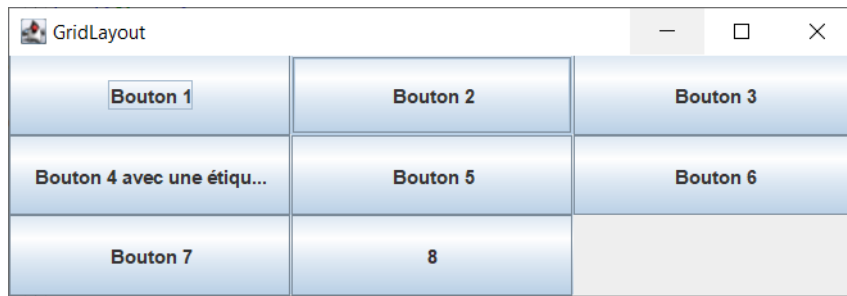


FIGURE 9 – Exemple avec GridLayout

l'utilisation de multiple fenêtres.

Méthodes couramment utilisées :

- `next(Container)` qui passe à la prochaine carte du conteneur ;
- `previous(Container)` qui retourne à la carte précédente du conteneur ;
- `first(Container)` qui retourne à la première carte du conteneur ;
- `last(Container)` qui passe à la dernière carte du conteneur ;
- `show(Container, String)` qui passe à la carte spécifiée avec le nom donné.